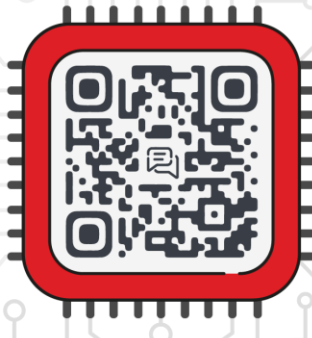




نظري

11

14

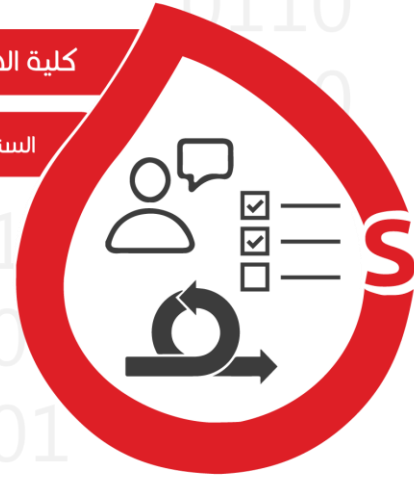
2100 sp

كلية الهندسة المعلوماتية

السنة الرابعة - قسم البرمجيات

Software Metrics

د. سعيد أبو طراب



محتوى مجاني غير مخصص للبيع التجاري
RB Informatics; 17/06/2026

هندسة برمجيات ٢

القياس وأهميته في هندسة البرمجيات

إن اتخاذ القرارات الخاطئة في الحياة العامة أو في علم هندسة البرمجيات يعود دائماً إلى الاعتماد على معلومات خاطئة. ولتجنب ذلك والتأكد من صدق وصحة المعلومات التي نمتلكها، لا بد من القيام بعمليات قياس دقيقة: فكل معلومة موثوقة تحتاج إلى أداة قياس ومقياس مناسب.

يهتم قياس البرمجيات (Software Measurement) باستنتاج قيمة رقمية لسمة معينة يمتلكها منتج برمجي، أو عملية برمجية، أو مشروع برمجي. هذا القياس هو ما يسمح للمهندسين والمدراء بإجراء مقارنات موضوعية بين التقنيات والعمليات المختلفة. ورغم الأهمية البالغة لبرامج القياس ودخولها في العديد من الشركات، إلا أن معظم المنظمات لا تزال لا تستخدم نظاماً منهجياً ثابتاً للقياس، كما أن المعايير المتبعة والمستقرة في هذا المجال ما زالت قليلة.

ملاحظة: في معظم المشاريع البرمجية يتم استخدام Software Metrics للتحقق من المتطلبات غير الوظيفية، ويتم استخدام ال Black Box Testing للتحقق من المتطلبات الوظيفية.

لماذا نقوم بعملية القياس (Goals of Measurement) ؟

نقوم بالقياس لتحقيق الأهداف الاستراتيجية التالية في إدارة المشاريع:

1. تقييم حالة المشروع بشكل مستمر: لمعرفة أين يقف المشروع حالياً مقارنة بالخطة الموضوعية.
2. السيطرة على المخاطر المحتملة: دراسة المخاطر قبل حدوثها والتحكم بها.
3. الكشف المبكر عن المشكلات الحرجة ("Critical") : اكتشاف الأخطاء في مراحلها الأولى قبل أن تتفاقم وتصل إلى مراحل متقدمة يصعب علاجها.
4. ضبط سير العمل: توجيه الفريق وضمان عدم الانحراف عن المسار المحدد.
5. مراقبة جودة منتجات العمل البرمجي: تقييم قدرة فريق المشروع على تقديم منتج ذي جودة عالية.

أجمل ما في العلم أنه لا يمكن لأحد أخذه منك



المفاتيح الأساسية لقياس البرمجيات (Software Measurement Keys)

للبدء في القياس، يجب اختيار مقياس مناسب لما نريد قياسه، وربطه بقيم صحيحة تضمن تقديم مؤشرات قابلة للقراءة. وتتكون منظومة القياس من ثلاثة مفاهيم جوهرية:

1. **القياس (Measure)** : يقدم مؤشراً كمياً لمدى أو قيمة أو حجم أو سعة بعض خصائص المنتج أو العملية (مثل: عدد أسطر الكود في برنامج).
2. **المقياس (Metric)** : هو مقياس كمي للدرجة التي يمتلك بها النظام، أو المكون (Component)، أو العملية (Process) سمة أو خاصية معينة (مثل: مؤشر الضبابية The Fog Index، أو عدد الأيام/الأشخاص المطلوبة لتطوير مكون برمجي).
3. **المؤشر (Indicator)** : هو مقياس أو مجموعة (تركيبية) من المقاييس التي توفر إحصاءات ورؤى حول العملية البرمجية (Software Process)، أو المشروع (Software Project)، أو المنتج (Software Product)، مما يساعد الإدارة على اتخاذ قرارات توجيهية.

تصنيف مقاييس البرمجيات (Classifications of Metrics)

تنقسم مقاييس البرمجيات من حيث طريقة الحصول عليها إلى قسمين رئيسيين:

1. المقاييس المباشرة (Direct Measures)

هي المقاييس التي نطبق فيها أداة القياس لنحصل مباشرة على قيم ذات معنى عن طريق الملاحظة أو العد المباشر. **أمثلة عليها:** طول الكود المصدري (Length of source-code)، مدة العملية الزمنية (Duration of process)، عدد العيوب المكتشفة (Number of defects discovered).

2. المقاييس غير المباشرة (Indirect Measures)

هي المقاييس التي لا نحصل عليها مباشرة عبر العد، بل يتم حسابها من خلال علاقات رياضية تربطها بمقاييس مباشرة أو غير مباشرة أخرى. وهي تربط صفتين أو أكثر لإعطاء قيمة كمية لسمة لا يمكن قياسها بالعد المباشر (مثل قياس الأخلاقيات بربطها بعدد ساعات الدوام)، أي نربط ال Indirect Measures بأكثر من Direct Measure ويكون بينهم علاقة، هذه العلاقة هي ما نسميه بال Indirect Measure. **أمثلة عليها:** كثافة عيوب المكون البرمجي (Module Defect Density)، وتحسب بالقانون التالي:

$$\text{Module Defect Density} = \text{Number of defects discovered} / \text{Length of source}$$

مثال تقريبي من الفيزياء:

قياس درجة الحرارة (Temperature) الذي يتم اشتقاقه هندسياً من طول عمود السائل داخل الميزان

الفرضيات والمبادئ الحاكمة للمقاييس (Metrics Assumptions & Principles)

1. مبادئ القياس الفني (Technical Principles)

حتى تكون عملية القياس صحيحة ومثمرة، يجب الالتزام بالمبادئ التالية:

1. تحديد أهداف القياس بوضوح تام قبل البدء في جمع البيانات.
2. تعريف كل مقياس تقني بطريقة محددة لا شك فيها ولا تحتمل التأويل (Unambiguous manner).
3. اشتقاق المقاييس استناداً إلى نظرية علمية صالحة ومجربة في مجال التطبيق (مثال: مقاييس التصميم يجب أن تعتمد على مفاهيم ومبادئ التصميم الأساسية لتوضح مدى وجود سمة مرغوبة في التصميم).

2. الفرضيات الأساسية للمقاييس (Metrics Assumptions)

عند التعامل مع مقاييس البرمجيات، يجب إدراك القواعد والفرضيات التالية:

1. لا يمكن قياس البرمجيات بدقة مطلقة.
2. المقاييس تنقسم إلى:

سمات داخلية (Internal Attributes) :

وهي الخصائص التي يمكن قياسها وفحصها مباشرة من قبل المهندسين داخل الكود أو التصميم (مثل التعقيد، حجم الكود).

سمات خارجية (External Attributes) :

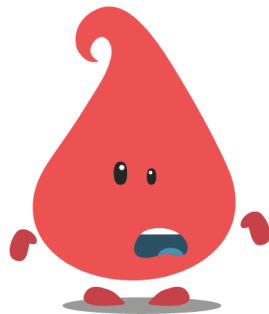
وهي الخصائص ذات الأهمية الكبرى للمستخدم أو الإدارة ولكن لا يمكن ملاحظتها أو قياسها إلا أثناء التشغيل أو من قبل شخص خارجي (مثل الاعتمادية، سهولة الفهم، القابلية للصيانة).

3. توجد علاقة وثيقة ومصاغة بشكل رسمي ومتحقق منه بين ما يمكن قياسه (السمات الداخلية) وما نريد معرفته (السمات الخارجية).

4. قد يكون من الصعب أحياناً ربط ما يمكن قياسه داخلياً بخصائص الجودة الخارجية المرغوبة بشكل مباشر.

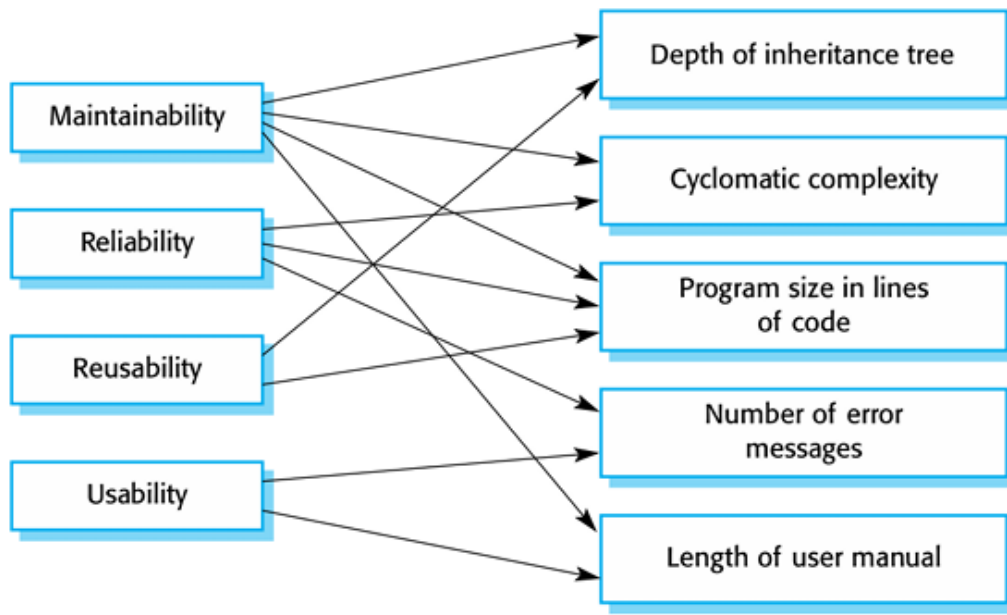
مثال توضيحي:

عند صناعة سيارة جديدة، فإن السرعة العالية ميزة خارجية مرغوبة (External Attribute) يلاحظها الزبون، ولكن لتحقيقها داخلياً يجب على المهندس التركيز على السمات الداخلية (Internal Attributes) مثل كيفية تصميم العجلات والمحرك بطريقة تتناسب مع تلك السرعة.



External quality attributes

Internal attributes



مراحل وخطوات دورة القياس (Measurement Process Steps)

تمر عملية تطبيق أي مقياس بخمس مراحل متسلسلة:

1. Formulation

2. Collection

3. Analysis

4. Interpretation

5. Feedback

- **التشكيل (Formulation):** اشتقاق وتشكيل المقياس البرمجي المناسب لطبيعة البرنامج الجاري فحصه، والتأكد مما إذا كان سيتم استخدام مقياس موجود مسبقاً أو ابتكار مقياس جديد يضمن إعطاء نتائج صحيحة ومناسبة للهدف.
- **التجميع (Collection):** الآلية والوسائل المستخدمة لجمع وتراكم البيانات المطلوبة لإيجاد وحساب المقاييس المصاغة.
- **التحليل (Analysis):** حساب قيم المقاييس الفعلية وتطبيق الأدوات الرياضية والإحصائية على البيانات المجمعة.
- **التفسير (Interpretation):** تقييم وفحص نتائج التحليل للحصول على رؤية واضحة ("Insight") حول جودة المنتج أو كفاءة العملية.
- **التغذية الراجعة (Feedback):** تقديم التوصيات والقرارات الناتجة عن عملية التفسير (بعض الإجراءات تستخدم لتطوير المشروع الحالي والبعض الآخر يتم استثماره كخبرات للمشاريع القادمة).

الأنواع الثلاثة الكبرى للمقاييس (Process, Product, & Project Metrics)

تقسم مقاييس البرمجيات حسب النطاق والهدف الأساسي إلى ثلاثة أنواع رئيسية:

1. أولاً: مقاييس العمليات (Process Metrics)

تهتم بقياس الفعالية والجودة العامة للعملية البرمجية (Software Process) والمنهجية المتبعة، ومعرفة مدى قدرتها على إنتاج برمجيات متكاملة ضمن الوقت والموارد المحددة. وهي تقيس كفاءة العملية بشكل غير مباشر.

■ المؤشرات التي يتم مراقبتها ضمن مقاييس العمليات:

- الوقت المستغرق لإتمام عملية معينة (Time taken) : قد يشمل الوقت الكلي، أو الوقت التقويمي (Calendar time)، أو الوقت الذي يقضيه مهندسون محددون على المهمة.
- الموارد المطلوبة للعملية (Resources required) : تشمل الجهد الكلي محسوباً بالأيام/الأشخاص (Person-days)، أو تكاليف السفر، أو موارد الكمبيوتر المستهلكة.
- عدد مرات حدوث حدث معين (Number of occurrences) : مثل عدد العيوب المكتشفة أثناء فحص الكود البرمجي (Code inspection)، عدد التغييرات المطلوبة في المتطلبات، أو عدد تقارير الأخطاء المرفوعة.

■ تصنيفات ومجالات إضافية للبيانات المجمعة في العمليات والمشاريع:

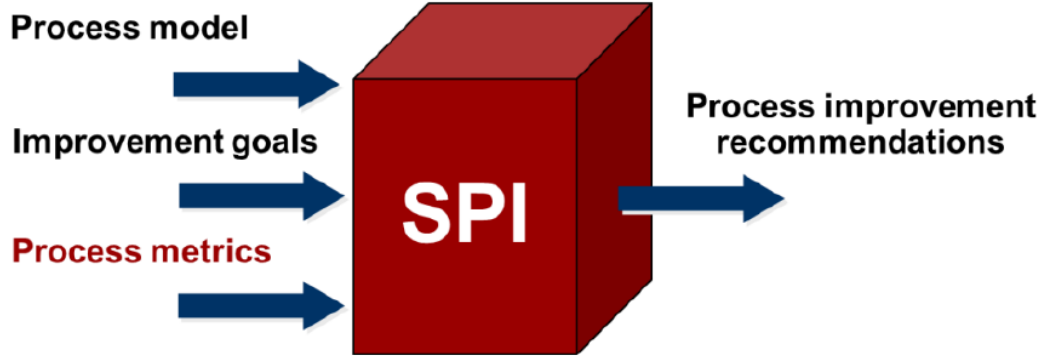
1. مقاييس متعلقة بالجودة (Quality-related) : التركيز الشديد على جودة منتجات العمل.
2. مقاييس متعلقة بالإنتاجية (Productivity-related) : ترتبط بالجهد المبذول لإنتاج برمجيات العمل.
3. بيانات ضمان الجودة الإحصائية (Statistical SQA data) : تصنيف الأخطاء وتحليلها إحصائياً.
4. كفاءة إزالة العيوب (Defect removal efficiency) : تقليل طول فترة بقاء الأخطاء وتلافي أثرها على حياة المشروع.
5. بيانات إعادة الاستخدام (Reuse Data) : قياس درجة استخدام المكونات البرمجية، وعدد المكونات المعزولة القابلة لإعادة الاستخدام في مشاريع مستقبلية لتقليل المخاطر والتكلفة.

■ الإرشادات الإدارية والأخلاقية لتطبيق مقاييس العمليات (Process Metrics Guidelines)

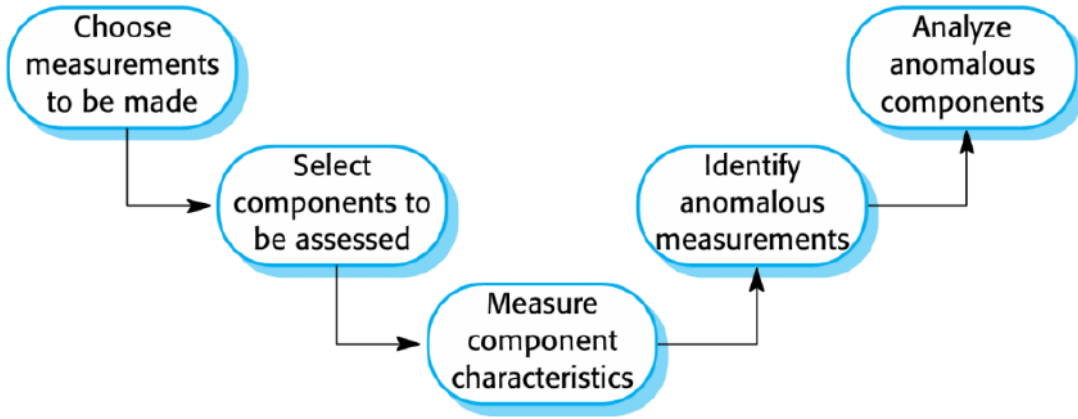
- استخدام الحس السليم ومراعاة الحساسية التنظيمية داخل الشركة عند تفسير وتحليل بيانات المقاييس.
- يجب تقديم التغذية الراجعة والنتائج بانتظام للأفراد والفرق الذين يجمعون هذه التدابير والمقاييس ليعرفوا قيمة عملهم.
- المنع البات لاستخدام المقاييس لتقييم الأفراد أو تهديدهم بيانات المقاييس التي تشير إلى وجود مشكلة في مكان ما أو كود معين يجب ألا تعتبر أمراً "سلبياً" أو تهمة تقصير، بل هي مجرد مؤشر مفيد لتحسين العملية البرمجية وتوجيه الدعم.

- العمل جنباً إلى جنب مع الممارسين والفرق الفنية لوضع أهداف واضحة ومقاييس مفهومة يتفق عليها الجميع قبل بدء المشروع.
- يجب عدم التركيز أو الهوس بمقياس واحد فقط (مثل عدد الأسطر أو سرعة التسليم) وإهمال المقاييس الهامة الأخرى (مثل الجودة والاعتمادية)؛ لأن مقياس الجودة الحقيقي يجب أن يكون قادراً على التنبؤ بجودة المنتج النهائي بشكل متوازن.

Software Process Improvement (SPI)



Process of Product Measurement



2. ثانياً: مقاييس المنتج (Product Metrics)

تستخدم لتقييم وفحص المنتج البرمجي نفسه. ويمكن استخدامها لإجراء تنبؤات عامة حول الصفات الخارجية للبرنامج. مقاييس المنتج لا يمكن أن تقاس إلا بعد الحصول على المنتج وهو يعتبر مؤشراً لجودة المنتج. تنقسم مقاييس المنتج البرمجي إلى قسمين رئيسيين:

1- المقاييس الديناميكية (Dynamic Metrics)

- يتم جمعها من خلال قياسات تجرى على البرنامج أثناء فترة تشغيله وتنفيذه الفعلي (In execution).
- ترتبط ارتباطاً وثيقاً ومباشراً بصفات الجودة الخارجية (External quality attributes)

فائدتها: تساعد على تقييم الكفاءة (Efficiency) والاعتمادية (Reliability)

مثال: يسهل قياس زمن استجابة النظام (Response time) كسمة للأداء، أو عدد مرات الفشل والانهيار (Number of failures) كسمة للاعتمادية.

2- المقاييس الساكنة (Static Metrics)

يتم جمعها من خلال قياسات تجرى على تمثيلات النظام البرمجي (الكود المصدري، التصميم، الوثائق) دون تشغيله، أي في وضع عدم التشغيل.

ترتبط بعلاقة غير مباشرة مع صفات الجودة، حيث يحتاج المهندس إلى اشتقاق العلاقة بين القياس الساكن والصفة الخارجية.

فائدتها: تساعد على تقييم التعقيد (Complexity)، وقابلية الفهم (Understandability)، وقابلية الصيانة (Maintainability).

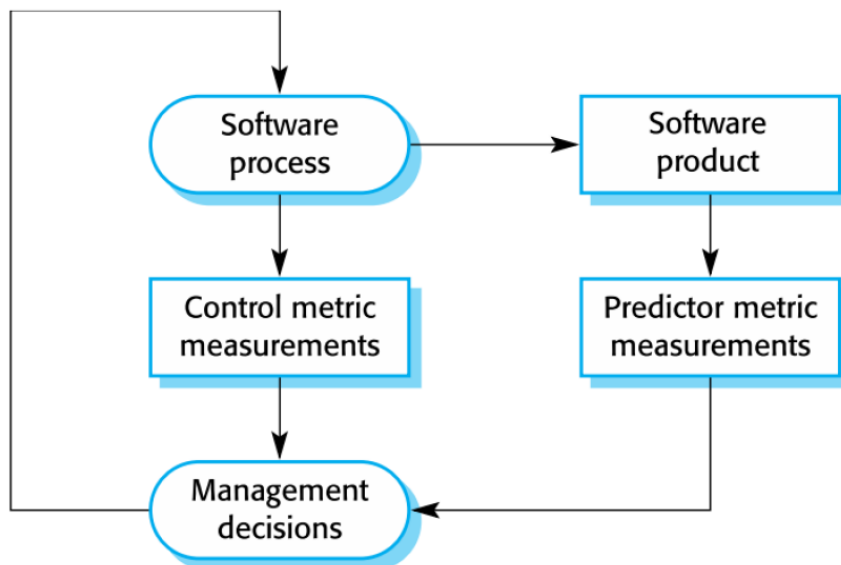
أمثلة على معايير المنتج الساكنة:

- مقياس التدفق الداخلي والخارجي (Fan-in / Fan-out) : الـ *Fan-in* يمثل عدد الـ Methods التي تقوم باستدعاء وظيفة معينة. الـ *Fan-out* يمثل عدد الوظائف الخارجية التي تقوم هذه الوظيفة باستدعائها.
- طول الكود البرمجي (Length of code) : عدد الأسطر.
- عمق الشروط المتداخلة (Depth of conditional nesting) : يقيس مدى تداخل جمل الشروط مثل (if-else) داخل الكود.

3. ثالثاً: مقاييس المشاريع (Project Metrics)

قياس مشروع ما إذا تم إنجازه ضمن الفترة المطلوبة، حيث نقول عن مشروع أنه ناجح إذا حقق العوائد المناسبة خلال فترة زمنية محددة، حيث يستخدم لتقليل جدول التطوير من خلال إجراء التعديلات اللازمة لتجنب التأخير والتخفيف من المشكلات والمخاطر المحتملة، ولتقييم جودة المنتج.

ملاحظة: إن الـ process metrics and project metrics متشابهان لحد كبير، والفرق أن الـ project عبارة عن مجموعة من الـ process.



الأبعاد الثلاثة لقياس أي مشروع (What Every Project Measures)

يقيس كل مشروع ثلاثة أبعاد أساسية:

1. المدخلات (Inputs) : مقياس الموارد المطلوبة لإنجاز العمل (مثل الأشخاص، الأدوات، الميزانية).
2. المخرجات (Outputs) : مقياس منتجات العمل البرمجي التي تم إنشاؤها وتطويرها أثناء عملية هندسة البرمجيات.
3. النتائج (Results) : المقياس التي تشير إلى مدى فعالية وجودة التسليم (Deliverables)

مقاييس المشاريع النموذجية والتقليدية (Typical Project Metrics)

- الجهد والوقت المستغرق لكل مهمة.
- عدد الأخطاء المكتشفة في كل ساعة مراجعة (Errors uncovered per review hour)
- التواريخ المجدولة للمواعيد النهائية مقابل التواريخ الفعلية التي تم تسليمها (Scheduled vs. actual milestone dates)
- عدد التغييرات المطلوبة (Changes) وخصائص وتأثيرات تلك التغييرات.
- توزيع الجهد على مختلف مهام هندسة البرمجيات.

ملاحظة: ال Project يتضمن ال process وال people, لذلك ينظر إلى ال project كعنصر داخلية كل واحدة على حدة.

منهجين رئيسيين لقياس الحجم والإنتاجية:

1. المنهج الأول: المقاييس الموجهة بالحجم (Size-Oriented Metrics)

تعتمد هذه المقاييس بشكل مباشر على حجم البرنامج أو النظام البرمجي الذي تم بناؤه. المقياس الأساسي المستخدم هنا هو عدد أسطر الكود (LOC - Lines of Code) أو بالآلاف (KLOC - Thousand Lines of Code).

تقييم مقياس عدد أسطر الكود (LOC / KLOC)

المميزات : سهل القياس بشكل آلي وحسابي.

العيوب:

- غير محدد أو معرّف بشكل جيد بالنسبة للغات البرمجة الحديثة.
- يُعتبر مؤشراً ضعيفاً جداً للإنتاجية لأنه يتجاهل تماماً إعادة استخدام الكود البرمجي ، تكرار الأكواد ، والفوائد الناتجة عن تحسين وإعادة التصميم.
- يقع في مفارقة لغات البرمجة: كلما كانت لغة البرمجة ذات مستوى منخفض، تطلب ذلك سطوراً أكثر، فيبدو المبرمج وكأنه أكثر إنتاجية!
- كلما كان المبرمج كثير الكلام في كوده زادت إنتاجيته ظاهرياً.



القوانين الحسابية للمقاييس الموجهة بالحجم (Size-Oriented Project Metrics)

تُحسب النسب والمؤشرات البرمجية بقسمة المتغيرات على حجم الكود (بالـ KLOC أو LOC) للتنبؤ بالتكلفة الإجمالية وصياغة التقارير:



- الأخطاء لكل ألف سطر كود Errors per KLOC
- العيوب لكل ألف سطر كود Defects per KLOC
- التكلفة المالية لكل سطر كود \$ per LOC
- صفحات التوثيق لكل ألف سطر كود Pages of documentation per KLOC
- عدد الأسطر المكتوبة لكل شخص في الشهر الواحد LOC per person-month
- الأخطاء لكل شخص في الشهر Errors per person-month
- الأخطاء المكتشفة لكل ساعة مراجعة وفحص Errors per review hour
- التكلفة المالية لكل صفحة توثيق مستندات \$ per page of documentation

2. المنهج الثاني: المقاييس الموجهة بالوظيفة (Function-Oriented Metrics)

تعتمد هذه المقاييس على استخدام نقطة الوظيفة (Function Point - FP) كعامل تسوية قياسي أو كمقياس مستقل للتعبير عن "حجم" ونطاق مواصفات المتطلبات، بمعزل عن لغة البرمجة المستخدمة.

مفهوم مقياس نقطة الوظيفة (Function Point Metric) :

يقوم الـ FP بقياس الوظائف الفعلية الملموسة التي يقدمها النظام البرمجي للمستخدم النهائي. ويتم اشتقاق قيمة الـ FP عبر علاقة تجريبية تعتمد على مقاييس مباشرة قابلة للعد في نطاق معلومات البرنامج (Information Domain)، بالإضافة إلى تقييم شامل لدرجة تعقيد البرنامج.

القيم الخمس لنطاق المعلومات (Information Domain Values) :

عند حساب الـ FP

أولاً: يتم حصر وعدّ العناصر الخمسة التالية:

1. عدد المدخلات الخارجية (Number of External Inputs - EIs) : البيانات أو الأوامر التي تدخل للنظام من الخارج.
2. عدد المخرجات الخارجية (Number of External Outputs - EOs) : التقارير أو شاشات البيانات التي تنتج خارج النظام.
3. عدد الاستفسارات الخارجية (Number of External Inquiries - EQs) : العمليات التي تتطلب طلباً فورياً للبيانات دون تعديل عليها.
4. عدد الملفات المنطقية الداخلية (Number of Internal Logical Files - ILFs) : قواعد البيانات أو التجمعات المنطقية التي يتحكم بها النظام داخلياً.

5. عدد ملفات الواجهات الخارجية (Number of External Interface Files - EIFs) : الملفات أو البيانات التي يتم تبادلها مع أنظمة برمجية أخرى.

ثم لكل عنصر من العناصر الخمسة السابقة وزن رياضي ثابت يعتمد على درجة تعقيده البرمجي (بسيط، متوسط، معقد) وفق الجدول التالي:

Information Domain Value	Count	Weighting factor				
		simple	average	complex		
External Inputs (EIs)		3	3	4	6	=
External Outputs (EOs)		3	4	5	7	=
External Inquiries (EQs)		3	3	4	6	=
Internal Logical Files (ILFs)		3	7	10	15	=
External Interface Files (EIFs)		3	5	7	10	=
Count total						→

ثانياً: معرفة العوامل التي تؤثر على النظام، وهي تتمثل ب 14 عامل، ولكل عامل درجة تأثير يتراوح بين 0 إلى 5، وتكون درجة الصغر عندما يكون العامل ليس موجوداً على النظام، أما درجة 5 يكون له استخدام كبير ضمن النظام 100٪، وهي:

1. Does the system require reliable backup and recovery?
2. Are data communications required?
3. Are there distributed processing functions?
4. Is performance critical?
5. System to be run in an existing, heavily utilized environment?
6. Does the system require on-line data entry?
7. On-line entry requires input over multiple screens or operations?
8. Are the master files updated on-line?
9. Are the inputs, outputs, files, or inquiries complex?
10. Is the internal processing complex?
11. Is the code designed to be reusable?
12. Are conversion and instillation included in the design?
13. Multiple installations in different organizations?
14. Is the application designed to facilitate change and ease-of-use?

$$FP = \sum (\text{count} \times \text{weight}) \times (0.65 + 0.01 \times N)$$

حيث إن:

$$\sum (\text{count} \times \text{weight})$$

هو المجموع الناتج عن ضرب عدد كل عنصر من عناصر نطاق المعلومات (المدخلات، المخرجات، الاستفسارات، الملفات) بالوزن الذي يناسب درجة تعقيده في الجدول.

N : هو حاصل جمع تأثير العوامل الـ 14 (العوامل العامة التي تؤثر على تعقيد النظام بيئياً وتقنياً).

الجزء $(0.65 + 0.01 \times N)$ يُعرف باسم مُضاعف التعقيد (Complexity Multiplier).

مثال توضيحي مصغر لحساب المجموع الموزون:

إذا كان لدينا في نظام ما أربع مدخلات خارجية ($EI = 4$) ؛ اثنان منها تعقيدهما بسيط، وواحد متوسط، وواحد معقد. بالرجوع لجدول الأوزان للمدخلات (بسيط=3، متوسط=4، معقد=6)، يتم حساب المجموع كالتالي:

$$\text{Sum for ELs} = (2 \times 3) + (1 \times 4) + (1 \times 6) = 6 + 4 + 6 = 16$$

يتم تكرار هذه العملية لبقية العناصر الخمسة ثم ضرب المجموع الكلي بمُضاعف التعقيد لنحصل على الـ FP النهائي.

القوانين والمؤشرات المبنية على نقاط الوظيفة (Function-Oriented Project Metrics)

- الأخطاء لكل نقطة وظيفة Errors per FP
- العيوب لكل نقطة وظيفة Defects per FP
- التكلفة المالية لكل نقطة وظيفة \$ per FP
- صفحات التوثيق لكل نقطة وظيفة Pages of documentation per FP
- نقاط الوظيفة المنجزة لكل شخص في الشهر FP per person-month



لترسم مستقبلك المشرق عليك ببذل الجهد والتعب
فالأحلام ليست مستحيلة، وبالجهد والاجتهاد تتحقق

مقارنة شاملة: نقاط الوظيفة مقابل أسطر الكود (FP vs. LOC)

توضح المقارنة التالية الفروقات الجوهرية والخصائص التفصيلية لكل منهما:

وجه المقارنة	أسطر الكود (LOC - Lines of Code)	نقاط الوظيفة (FP - Function Points)
مرحلة القياس الأولى	لا يمكن قياسها بدقة إلا بعد انتهاء كتابة الكود بالكامل	يمكن قياسها وحسابها مبكراً مباشرة بعد مرحلة التصميم
الاعتمادية على اللغة	تعتمد بشدة على نوع ولغة البرمجة ومستوى تعقيدها	مستقلة تماماً عن لغة البرمجة المستخدمة
التقدير والتحويل	قد يختلف عدد أسطر الكود بشكل كبير بالنسبة لنفس قيمة FP تبعاً للغة البرمجة وأسلوب التطوير.	يمكن استخدامها لتقدير عدد أسطر الكود (LOC) المتوقعة بناءً على متوسط عدد الأسطر لكل FP للغة معينة
الموضوعية والأتمتة	مقاييس موضوعية حسابية صلبة وسهلة القياس والأتمتة بواسطة أدوات الكمبيوتر	مقاييس ذاتية شخصية تعتمد بشكل كبير على تقدير وخبرة الشخص المُقدِّر، لا يمكن عدّها تلقائياً بشكل آلي.
إعادة الاستخدام (Reuse)	تتجاهل وتعمي الإدارة عن أثر إعادة الاستخدام والمكونات المكررة	تسهل بشكل كبير قياس ومعرفة مدى تأثير وأثر المكونات البرمجية المعاد استخدامها داخل النظام

مقاييس نماذج هندسة البرمجيات والتصميم (Design Model Metrics)

تنقسم مقاييس التصميم الفني إلى ثلاثة مستويات برمجية:

1. مقاييس نموذج المتطلبات (Requirements Model Metrics)

- المقاييس القائمة على الوظائف (Function-based Metrics): تستخدم الـ Function Point كمؤشر ومقياس لحجم المواصفات المطلوبة.
- مقاييس المواصفات (Specification Metrics): تستخدم كمؤشر لتقييم جودة المواصفات عن طريق عدّ وتصنيف عدد المتطلبات حسب نوعها وأهميتها.

2. مقاييس التصميم المعماري (Architectural Design Metrics)

تهتم هذه المقاييس بتقييم بنية وهيكلية النظام وتداخل الوحدات والمكونات المعمارية:

- **التعقيد الهيكلي (Structural Complexity)** : يُحسب كدالة رياضية تعتمد على التدفق الخارجي للمكونات البرمجية:

$$\text{Structural Complexity} = g(\text{fan} - \text{out})$$

- **تعقيد البيانات (Data Complexity)** : يُحسب كدالة تعتمد على عدد متغيرات المدخلات والمخرجات والتدفق الخارجي $f(\text{input \& output variables, fan-out})$
- **تعقيد النظام الكلي (System Complexity)** : حاصل دمج وتأثير التعقيد الهيكلي وتعقيد البيانات معاً $h(\text{structural \& data complexity})$
- **مقياس هينري وكافاناه (HK Metric)** : يقيس التعقيد المعماري الشامل كدالة رياضية تعتمد على قيم التدفق الداخلي والخارجي معاً (fan-in) و (fan-out)
- **مقاييس المورفولوجيا (Morphology Metrics)** : دالة تعتمد على حساب العدد الإجمالي للموديولات (Modules) والروابط والواجهات البينية الموجودة بين هذه الموديولات.

3. مقاييس التصميم على مستوى المكونات (Component-Level Design Metrics)

تركز على تفاصيل البناء الداخلي للمكون البرمجي الواحد:

- **مقاييس التماسك (Cohesion Metrics)** : تقيس مدى ارتباط الصف مع بعضها البعض، وهي دالة تعتمد على كائنات البيانات (Data objects) ومكان تعريفها.
- **مقاييس الاقتران (Coupling Metrics)** : تقيس مدى اعتماد المكونات البرمجية على بعضها البعض، وهي دالة تعتمد على متغيرات المدخلات والمخرجات، والمتغيرات العامة المشتركة والموديولات التي يتم استدعاؤها.
- **مقاييس التعقيد (Complexity Metrics)** : تم اقتراح الكثير، ومن أشهرها التعقيد الديكلواتي (Cyclomatic Complexity) المعتمد على تتبع مسارات التحكم في البرنامج.

4. مقاييس واجهة التصميم (Interface Design Metrics)

- **ملائمة التخطيط (Layout Appropriateness)** : مقياس مخصص لتقييم جودة تخطيط واجهة المستخدم، وهو وظيفة تعتمد على (Layout entities)، والموقع الجغرافي للعناصر على الشاشة، وكلفة الانتقالات بين الكيانات.

المشكلات والتحديات الحقيقية في علم القياس (Metrics Issues)

تواجه الإدارات عند تطبيق مقاييس البرمجيات ثلاثة تحديات وجوانب حرجية يجب التعامل معها بحذر:

1. **غموض القياس (Measurement Ambiguity) :** عند تجميع كميات كبيرة من البيانات الرقمية والكمية حول البرمجيات وعملياتها، فإن الأرقام وحدها لا تكفي؛ بل يجب تحليل هذه البيانات وفحص سياقها بدقة لفهم معناها الحقيقي والعميق.
2. **سياق البرمجيات وبيئة العمل (Software Context) :** العمليات والمنتجات البرمجية التي يجري قياسها ليست معزولة أبداً عن بيئتها المحيطة. إن بيئة العمل والشركات تتغير وتتطور باستمرار، ومن المستحيل تجنب هذه التغييرات في ممارسات العمل لمجرد الحفاظ على ثبات القياس، على الرغم من أن هذه التغييرات البيئية قد تجعل مقارنات البيانات غير صالحة.
3. **أتمتة القياس وأدواته (Measurement Automation) :**

- ✓ أن تكون الأدوات سهلة الاستخدام للغاية.
- ✓ أن تعمل الأدوات بسرعة فائقة وتنتج مخرجات وتقارير مختصرة ومركزة بدلاً من كميات كبيرة من المعلومات.
- ✓ أن تصنع الأدوات قياسات متعددة وتأخذ بعين الاعتبار أكبر عدد ممكن من المتغيرات والمعايير؛ لأنه من المستحيل التنبؤ مسبقاً بالأفكار التي قد تظهر فجأة وتفيد العمل.
- ✓ أن تكون أدوات تفاعلية (Interactive) تسمح للمدراء والمطورين باستكشاف وتتبع التحليلات.

إلى هنا نكون قد طوينا الصفحة الأخيرة من رحلتنا في هندسة البرمجيات 2

مادة لم تكن مجرد سطور للحفظ، بل فكراً هندسياً يبني الأنظمة وينظم الأكواد

تمنياتنا لكم بالتوفيق والوصول للعلامة الكاملة!

